

BaseCode 的说明

新版的 BaseCode 支持界面自己定义，一共 6 个页面，每个页面 6 个功能。36 个位置，可以在功能列表里挑选想要的功能（瞄准镜单独做的，主菜单里不能选），支持菜单倒装，支持黑白切换。

同时，快捷键，也支持在功能列表里选择功能，包括瞄准镜（瞄准镜做了两种，第一种 4 个小图标的，第二种是 6 个大图标的）

1. SG_OSD_FuncDefine.h 定义了默认的 OSD 界面。
2. SG_OSD_UserInclude.h 定义了默认的功能。

这 2 个文件，建议大家不要去动，如果想修改某个默认值，请新建一个 xx_OSD_UserInclude.h，比如我的就是 LuoGang_OSD_UserInclude.h

```
// ID Code      : LuoGang_OSD_UserInclude.h No.0000      //IDO
// Update Note  : 相关定义都在 SG_OSD_UserInclude.h
//-----
#if(Project_ID == ID_COMMON)
    #define _LOGO_TYPE                      _REALTEK_LOGO

    #define _OSD_RECALL_FUNC                _ON //退出菜单是否记忆前面的状态

    #if(_PROJECT != _RL6463_PROJECT)
        #define _DEF_P1_PICTURE_ITEM4      _MENU_FUN_HDR
    #endif
#elif(Project_ID == ID_SG_2795_LM270WR2_TEST)
    #define _SOURCE_AUTO_ON                _ON //信号切换是否带auto

    #define _DEF_BACKLIGHT                  20

    #define _CT_7500_5800_sRGB_SUPPRT      _ON //色温是否带7500.5800.sRGB
#elif(Project_ID == ID_SG_2785_LM270WQ4_TEST)

    #define _OSD_RECALL_FUNC                _ON //退出菜单是否记忆前面的状态

    #define _DEF_FACTORY_MODE              _OFF

    #define _BACKLIGHT_MAX                  230

    #define _BACKLIGHT_MIN                  30

    #define _DEF_BACKLIGHT                  70

    #define _BACKLIGHT_DEF_PWM              0x90
#endif
```

3. 请再新建一个 xx_ProjectDefine.h，比如我的就是 LuoGang_ProjectDefine.h

注意：

xx_ProjectDefine.h 和 xx_OSD_UserInclude.h 请自己保留，Project_ID 如何命名自己选择，只要自己能分辨。我定义一个 SG_ProjectDefine.h，只是为了方便切换工程时，不用每次从头编译。

```
_MENU_FUN_BACKLIGHT,
_MENU_FUN_CONTRAST,
_MENU_FUN_BRIGHTNESS,
_MENU_FUN_SHARPNESS,
_MENU_FUN_ECO,
_MENU_FUN_DCR,
#if(_HDR10_SUPPORT == _ON)
_MENU_FUN_HDR,
#endif
_MENU_FUN_VGA_HPOSITION,
_MENU_FUN_VGA_VPOSITION,
_MENU_FUN_VGA_CLK,
_MENU_FUN_VGA_PHASE,
_MENU_FUN_ASPECT,

_MENU_FUN_COLORTEMP,
_MENU_FUN_R,
_MENU_FUN_G,
_MENU_FUN_B,

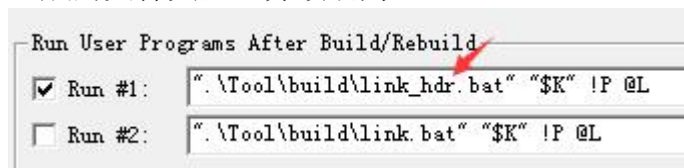
_MENU_FUN_LANGUAGE,
_MENU_FUN_OSD_HPOSITION,
_MENU_FUN_OSD_VPOSITION,
_MENU_FUN_OSD_TIME,
_MENU_FUN_OSD_TRANS,

_MENU_FUN_AUTO_ADJUST,
_MENU_FUN_AUTO_COLOR,
_MENU_FUN_RESET,

_MENU_FUN_SIGNAL,
_MENU_FUN_LOWBLUE,
_MENU_FUN_AUDIOIMUTE,
_MENU_FUN_AUDIOVOLUME,
_MENU_FUN_FREESYNC,
_MENU_FUN_OD,
_MENU_FUN_POWER_OFF,
_MENU_FUN_LED,
_MENU_FUN_HOTLEFT,
_MENU_FUN_HOTRIGHT,

_MENU_NULL,
/**/ *****Spacil Hotk
_HOT_KEY_LOS_TYPE1,
_HOT_KEY_LOS_TYPE2,
```

4. 如果带 HDR 功能，只需要用 link_hdr.bat 即可，并且把 hdr 数据放在 bin 文件生成的文件夹里，并改名为 HDR.bin



link_hdr.bat 相对于 link.bat 也只是多传了 2 个参数而已，一个是 hdr 文件的名字，一个是 hdr 存放在哪个 bank。

```
@REM ".\Tool\build\link.bat" "$K" !P @L
@set PATH="%1C51\bin";%PATH%
@Tool\build\auto_link.exe --uv "%2" --target "%3" --opt 3 --sort --hex2bin ".\HexToBin.exe"

if exist .\Output\%3\%3_WithChkSum.bin del .\Output\%3\%3_WithChkSum.bin
.\AddChecksum.exe .\Output\%3\%3.bin .\Output\%3\%3_WithChkSum.bin BANK1 .\Output\%3\HDR.bin HdrBank14
if exist .\Output\%3\%3.bin del .\Output\%3\%3.bin
```

它会把数据加载到指定的 bank，并计算 checksum，保存到第 4 个参数（bank1）的 0xffff 和 0xffff，code 里已经做好读取的函数。

/******

以上部分是对 osd 的简单介绍，下面对于功能的实现简单说一下，方便大家理解。

```
BYTE idata OsdStateDefine[6][6]={
```

```

{ // 1
    _MENU_FUN_BACKLIGHT,
    _MENU_FUN_CONTRAST,
    _MENU_FUN_SHARPNESS,
    _MENU_FUN_ECO,
    _MENU_FUN_DCR,
    _MENU_NULL
},

```

```
// 2
_MENU_FUN_VGA_HPOSITON,
_MENU_FUN_VGA_VPOSITON,
_MENU_FUN_VGA_CLK,
_MENU_FUN_VGA_PHASE,
_MENU_FUN_ASPECT,
_MENU_NULL
},
```

```

{ // 3
    _MENU_FUN_COLORTEMP,
    _MENU_FUN_R,
    _MENU_FUN_G,
    _MENU_FUN_B,
    _MENU_FUN_HOTLEFT,
    _MENU_FUN_HOTRIGHT
},

```

```
// 4
_MENU_FUN_LANGUAGE,
_MENU_FUN_OSD_HPOSITON,
_MENU_FUN_OSD_VPOSITON,
_MENU_FUN_OSD_TIME,
_MENU_FUN_OSD_TRANS,
_MENU_NULL
},
```

```
// 5
_MENU_FUN_AUTO_ADJUST,
_MENU_FUN_AUTO_COLOR,
_MENU_FUN_RESET,
_MENU_NULL,
_MENU_NULL,
```

2. 但是有个问题点：比如 ECO 不是标准的时候，要把 BACKLIGHT、CONTRAST、BRIGHTNESS、SHARPNESS 屏蔽掉，OSD 要把他们打灰，同时光标移动的时候，不能选择。我们需要找到它在哪个位置，所以：

就定义了 OSD ITEM BASE，它记录了每个功能在 OSD 中的位置

```
BYTE idata OSD ITEM BASE[ MENU NULL - MENU FUN BACKLIGHT] = {0};
```

还定义了 `OSD_MASK[6]`，它记录了，`OSD` 中那些位置被屏蔽了，开始我是定义了一个 `OSD_MASK[6][6]` 的二维数组，这样看起来更直观，但是 `RL6463` 的工程 `RAM` 区太小，为了兼容，用了一个 `OSD_MASK[6]` 的一维数组，比如 `OSD_MASK[0]` 记录了，第 0 个界面那些项被屏蔽了，它的第 0 个 bit 对应第 0 项，第 1 个 bit 对应第 1 项，以此类推。

```
BYTE idata OSD_MASK[6] = {0};
```

还定义了 2 个全局变量，他们记录了，OSD 的当前位置

```
BYTE idata OSD_MAIN_MENU_ITEM = 0;
BYTE idata OSD_SubMenuCur = 0;
```

最后定义了一个 OSD_ITEM_MASK[6] 的数组，他记录了哪些功能被屏蔽掉了，相对于 OSD_MASK[6]，他们的区别在于 OSD_MASK[6] 记录的是哪些位置被屏蔽。

```
BYTE idata OSD_ITEM_MASK[6] = {0};
```

一开始，它的大小也是跟着功能走的，还是为了节省空间，它的每一个 BYTE 存放了 8 种功能，比如 OSD_ITEM_MASK[0]，存放了功能列表的前 8 种：BACKLIGHT、CONTRAST 等等。

```
BYTE idata OSD_ITEM_MASK[_MENU_NULL - _MENU_FUN_BACKLIGHT] = {0};
```

这个数组目前只给 快捷键 使用，在选择快捷键的时候，屏蔽项不能选。比如，ECO 不是标准时，亮度、对比度等选项就不能选择，如果快捷键已经选了亮度，再调节 ECO，则会打灰，且不能调节，这都需要用到这个数组。

最后增加一个条件，主菜单没有的选项，快捷键也不让选：

```
for(i = _MENU_FUN_BACKLIGHT; i < _MENU_NULL; i++)
{
    if(OSD_ITEM(i) == 0xff)
    {
        OSD_ITEM_MASK_SET(i);
    }
}
```

左右快捷键的默认功能，就是左右快捷键的功能选择，送样机的时候方便客户，烧完软件，自己选择想要的快捷键。